

Task description

I need to build small Security Operation Center style by using two ubuntu machines. The first machine is used to create Wazuh SIEM server, the second one was used to create DVWA target system, where I also have Wazuh agent. The main goal was to forward logs from the DVWA to Wazuh machine, observe web attacks and monitor a honeypot file, and have automate notifications through a telegram bot.

First step

I created two virtual machines ubuntu

VM- wazuh-server

VM 2- dvwa-agent

First, I entered and checked their ip to see that they are in the same host.

Wazuh server: 192.168.56.103

DVWA agent: 192.168.56.101

Second step

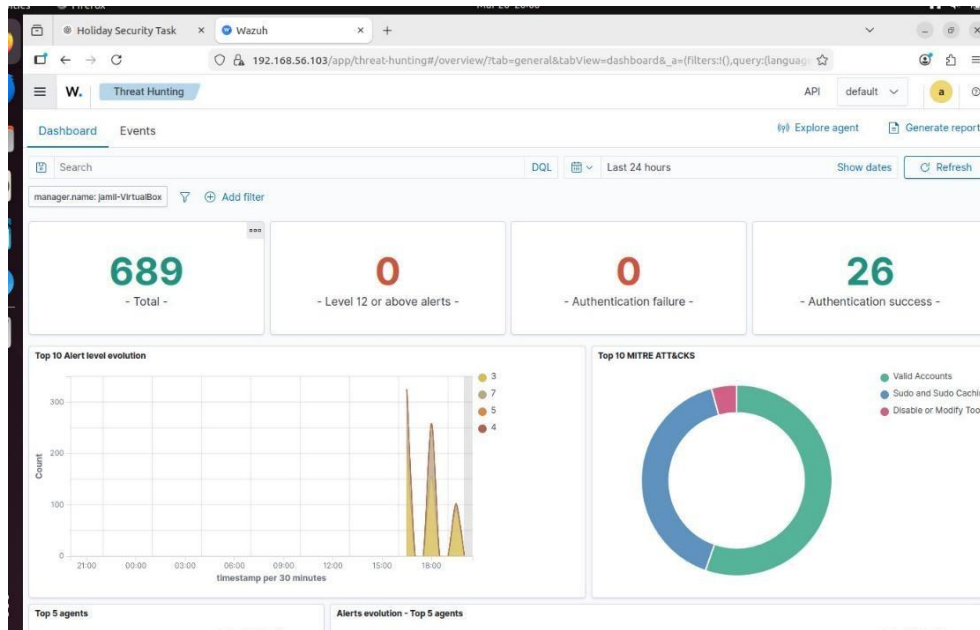
I downloaded Wazuh on first machine with these commands:

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
```

```
curl -sO https://packages.wazuh.com/4.14/config.yml
```

I had problem with disk space so I could not install it, from settings of VM I increased from 25gb to 50 gb. After that I successfully downloaded Wazuh with this command: `sudo bash wazuh-install.sh -a`

And finally I entered wazuh from here <https://192.168.56.103> and got access to SIEM dashboard.



Third step

The second machine I used to download vulnerable web application DVWA

I installed required packages with these commands:

```
sudo apt update
```

```
sudo apt install -y apache2 mariadb-server php php-mysql php-gd libapache2-mod-php git
```

Downloading DVWA:

```
cd /var/www/html
```

```
sudo git clone https://github.com/digininja/DVWA.git
```

Created the dvwa configuration file:

```
cd /var/www/html/DVWA
```

```
sudo cp config/config.inc.php.dist config/config.inc.php
```

Created also the database with `sudo mysql` and added these:

```
create database dvwa;
```

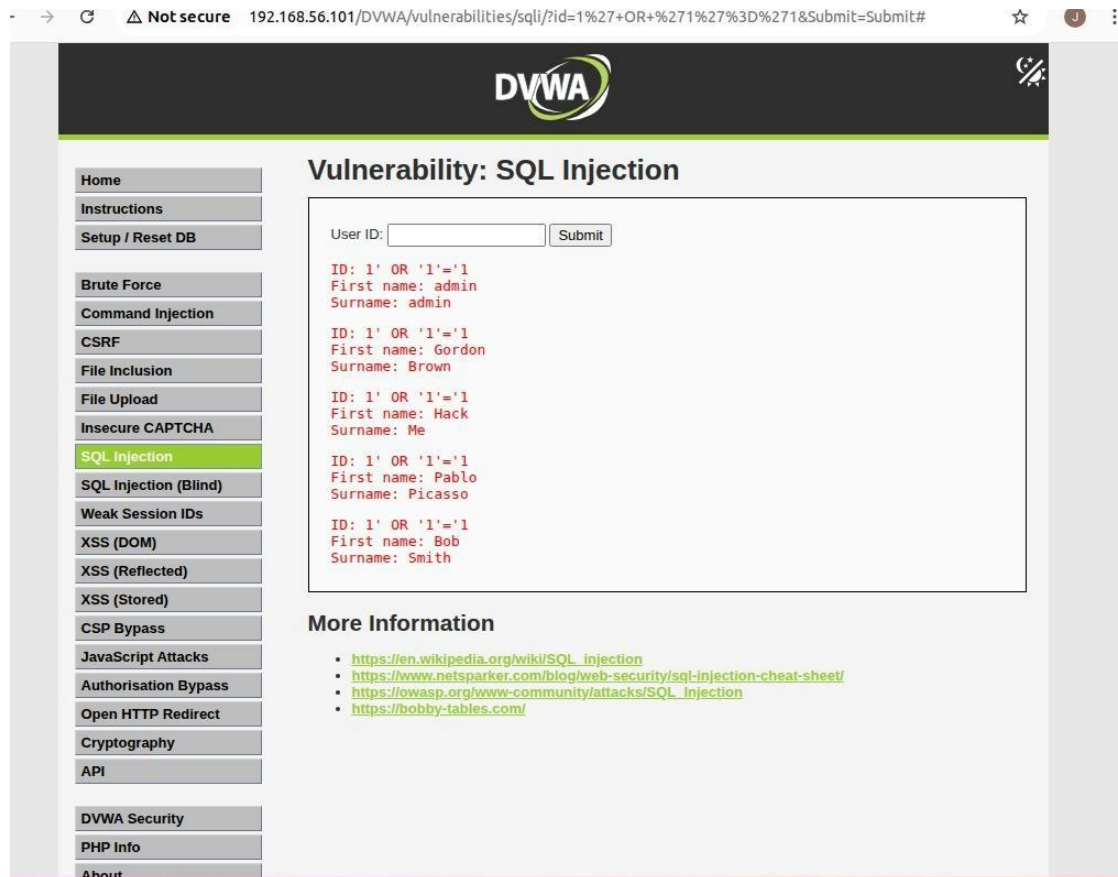
create user 'dvwa'@'localhost' identified by 'p@ssw0rd';

grant all on dvwa.* to 'dvwa'@'localhost';

flush privileges;

exit;

Finally I opened dvwa in browser by searching <http://1G2.168.56.101/DVWA> (i used default login to access



Fourth step

I started to download wazuh agent to “dvwa-agent” machine to be able to send endpoint logs to the SIEM server (Wazuh)

To download I used:

```
sudo apt install -y gnupg apt-transport-https
```

```
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo gpg --no-default-keyring --keyring /usr/share/keyrings/wazuh.gpg --import
```

```
echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg]  
https://packages.wazuh.com/4.x/apt/ stable main" | sudo tee  
/etc/apt/sources.list.d/wazuh.list
```

```
sudo apt update
```

```
sudo WAZUH_MANAGER='1G2.168.56.103' apt install -y wazuh-agent
```

Then I edited file with this command:

sudo nano /var/ossec/etc/ossec.conf - Inside of it I added these important sections like Destination IP (where I have wazuh) and port number

```
<client>
```

```
<server>
```

```
<address>1G2.168.56.103</address>
```

```
<port>1514</port>
```

```
<protocol>tcp</protocol>
```

```
</server>
```

```
<config-profile>ubuntu, ubuntu22, ubuntu22.04</config-profile>
```

```
<notify_time>20</notify_time>
```

```
<time-reconnect>60</time-reconnect>
```

```
<auto_restart>yes</auto_restart>
```

```
<crypto_method>aes</crypto_method>
```

```
</client>
```

Finally, I started the agent.

Step Five

I configured log collection from endpoint. I did it to be able to collect Apache and audit logs.

The following blocks were added to `/var/ossec/etc/ossec.conf` on the dvwa-agent machine, inside `<ossec_config>` but outside `<client>`:

```
<localfile>  
  <location>/var/log/apache2/access.log</location>  
  <log_format>apache</log_format>  
</localfile>
```

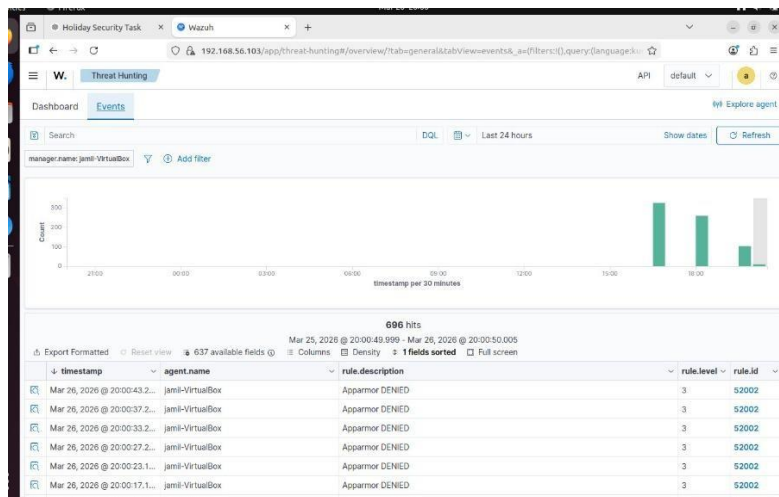
```
<localfile>  
  <location>/var/log/apache2/error.log</location>  
  <log_format>syslog</log_format>  
</localfile>
```

```
<localfile>  
  <location>/var/log/audit/audit.log</location>  
  <log_format>audit</log_format>  
</localfile>
```

After that I restarted wazuh-agent and did a test to see if everything works by using these commands:

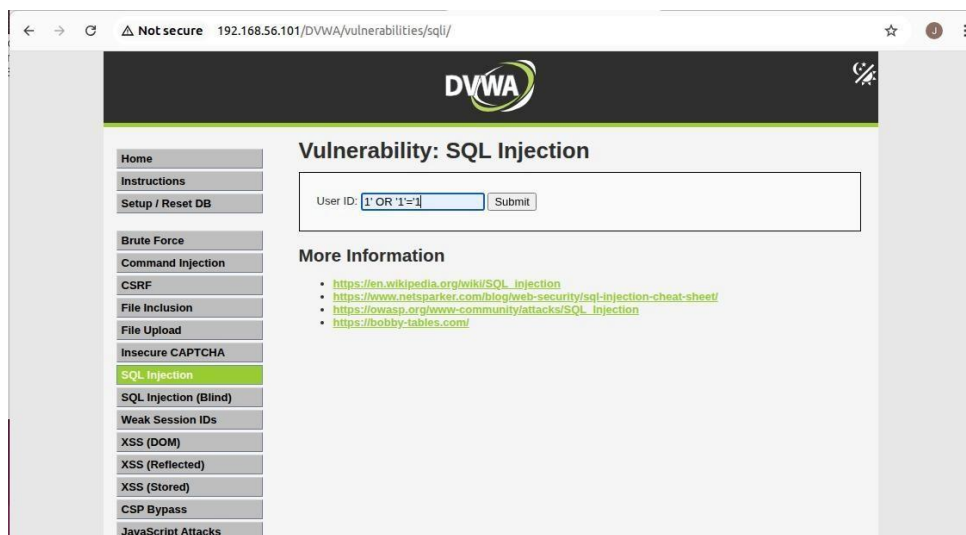
```
curl http://127.0.0.1/DVWA/login.php  
sudo tail -n 5 /var/log/apache2/access.log
```

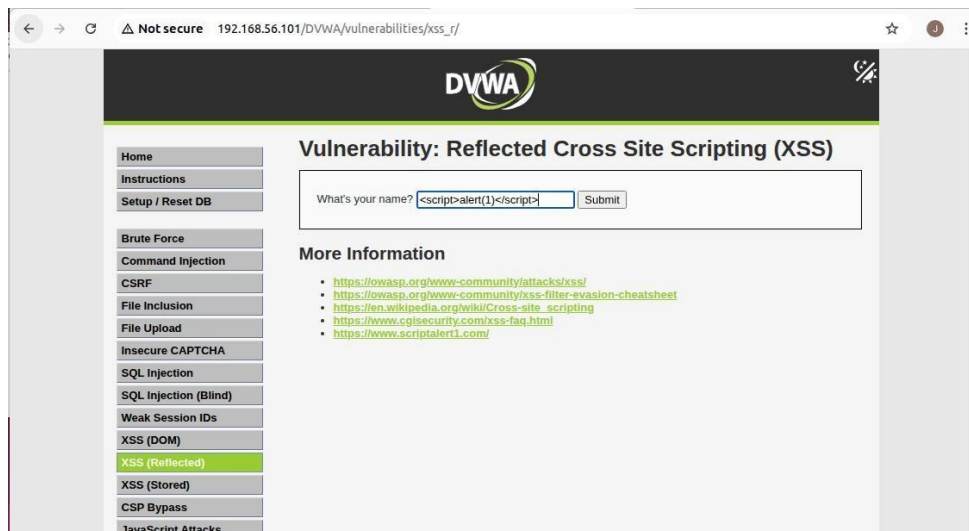
On Wazuh dashboard I checked if it works from Threat Hunting - Events. It worked.



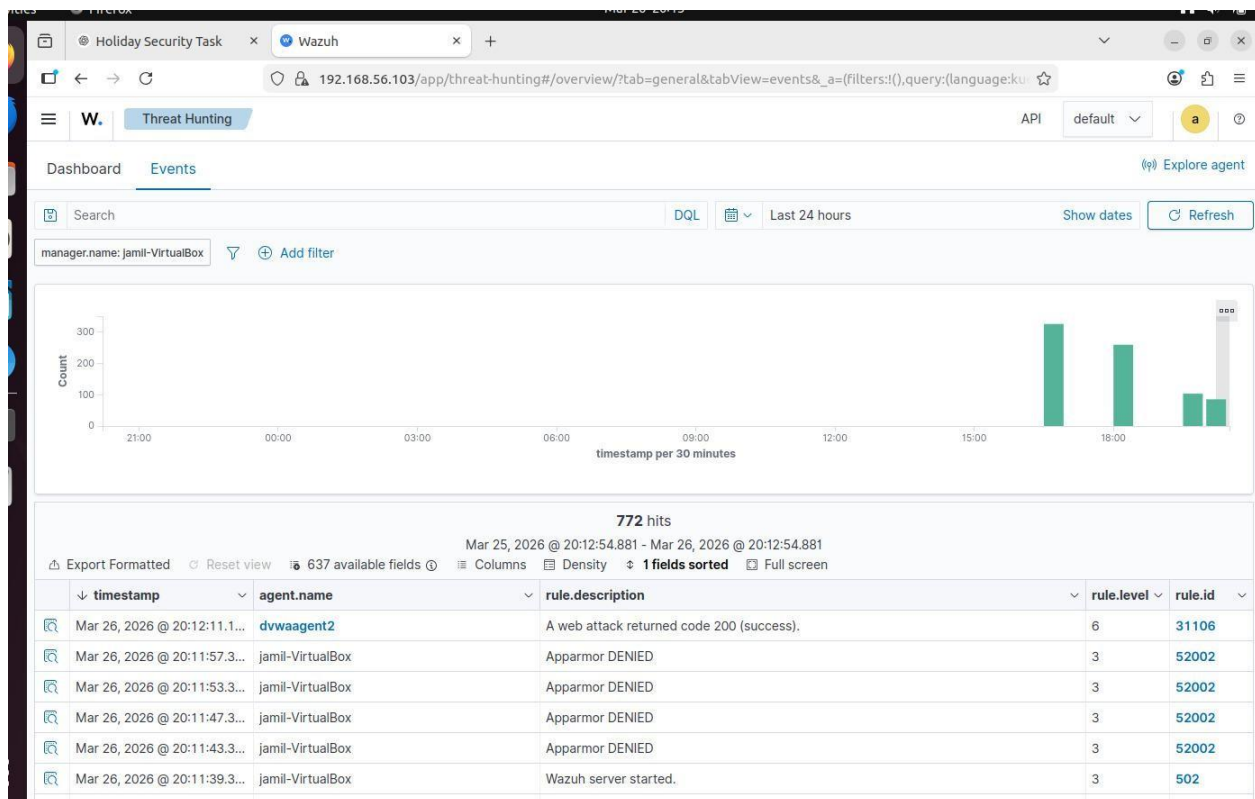
Step Six

Now im checking web attacks from DVWA. First, I did SQL injection, by submitting '1' OR '1'='1. After that I did reflected XSS by writing this script <script>alert(1)</script>.





It generated A web attack successful code 200, so the attack was successful and visible in the SIEM



Step Seven

From now im creating honeypot file on dvwa-agent. Here first I need to install auditd with these commands:

```
sudo apt update
```

```
sudo apt install -y auditd audispd-plugins
```

```
sudo systemctl enable auditd
```

```
sudo systemctl start auditd
```

After that im creating the honeypot file:

```
sudo mkdir -p /opt/honey
```

```
echo "TOP SECRET FILE" | sudo tee /opt/honey/token.txt
```

```
sudo chmod 600 /opt/honey/token.txt
```

Then I create audit rules:

```
echo '-w /opt/honey/token.txt -p rwa -k honeypot' | sudo tee  
/etc/audit/rules.d/honeypot.rules
```

```
sudo augenrules -load
```

And finally, I triggered the honeypot file to see alert in Wazuh:

```
sudo cat /opt/honey/token.txt
```

It worked out

Step Eight

In this step I was creating wazuh custom rules.

The file was edited using:

```
sudo nano /var/ossec/etc/rules/local_rules.xml
```

DVWA rule attempt

The following type of rules were tested:

```
<group name="dvwa_attacks,web,">
```

```
<rule id="100100" level="10">  
  <decoded_as>web-accesslog</decoded_as>  
  <match>select|union|information_schema|or</match>  
  <description>SQL Injection detected (DVWA)</description>  
  <group>web,sqli,dvwa,</group>  
</rule>
```

```
<rule id="100101" level="10">  
  <decoded_as>web-accesslog</decoded_as>  
  <match>script|alert|onerror|javascript:</match>  
  <description>XSS attack detected (DVWA)</description>  
  <group>web,xss,dvwa,</group>  
</rule>
```

```
</group>
```

Honeytoken rule attempt

```
<group name="honeypot,">  
  <rule id="100200" level="12">  
    <match>honeypot</match>  
    <description>Honeytoken file accessed</description>  
  </rule>  
</group>
```

The Wazuh manager was restarted after editing:

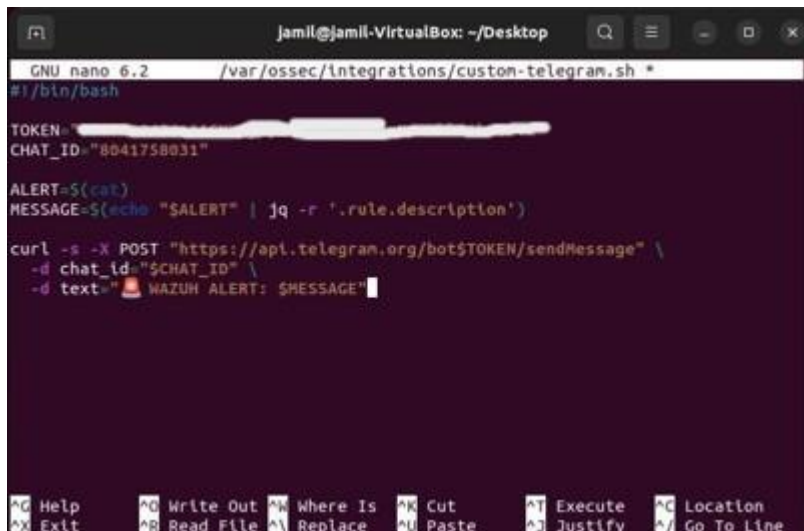
```
sudo systemctl restart wazuh-manager
```

```
sudo systemctl status wazuh-manager --no-pager
```

Step Nine

Finally, I need to create telegram bot, to get notifications from alerts. I used BotFather to create Wazuh alerts. BotFather gave me a TOKEN. I entered a link with that TOKEN <https://api.telegram.org/bot<TOKEN>/getUpdates> and there i saw the chat ID.

On Wazuh server I created script in `sudo nano /var/ossec/integrations/custom-telegram` and wrote this inside:

A screenshot of a terminal window titled 'jamil@jamil-VirtualBox: ~/Desktop'. The window shows the GNU nano 6.2 editor editing the file '/var/ossec/integrations/custom-telegram.sh'. The script content is as follows:

```
#!/bin/bash
TOKEN="XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
CHAT_ID="8041758031"

ALERT=$(cat)
MESSAGE=$(echo "$ALERT" | jq -r '.rule.description')

curl -s -X POST "https://api.telegram.org/bot$TOKEN/sendMessage" \
-d chat_id="$CHAT_ID" \
-d text="🔥 WAZUH ALERT: $MESSAGE"
```

The terminal window has a dark background and a light-colored text. The bottom of the window shows a status bar with various keyboard shortcuts like 'Help', 'Exit', 'Write Out', 'Read File', 'Where Is', 'Replace', 'Cut', 'Paste', 'Execute', 'Justify', 'Location', and 'Go To Line'.

A manual Telegram API test was first performed successfully:

```
curl -s -X POST "https://api.telegram.org/bot<TOKEN>/sendMessage" \
-d chat_id="8041758031" \
-d text="TEST MESSAGE"
```

After the Wazuh integration was enabled, alert-generating actions produced Telegram notifications.



Conclusion

This project successfully demonstrated the deployment of a basic SIEM monitoring lab using Wazuh. The lab included a monitored vulnerable web application, endpoint log forwarding, audit-based file monitoring, and real-time Telegram notifications

Additional Task

I need to create Third VM and use it for suricata IDS, so on the third machine I created suricata, from another machine I sent Nmap and checked if I got logs to ids and it worked.

After that on suricata machine I uploaded wazuh agent to be able to connect suricata to wazuh. Finally I successfully connected suricata to Wazuh and I saw nmap attacks in my dashboard.

The screenshot shows the Wazuh Threat Hunting dashboard in a Firefox browser window. The dashboard displays a list of alerts with columns for timestamp, agent name, and rule description. A detailed view of a specific alert is shown on the right, displaying its JSON structure.

timestamp	agent.name	rule.description
Mar 31, 2026 @ 12:42:39.5...	jamil-VirtualBox	PAM: Login sess
Mar 31, 2026 @ 12:42:39.5...	jamil-VirtualBox	PAM: Login sess
Mar 31, 2026 @ 12:42:35.5...	jamil-VirtualBox	Successful sudo
Mar 31, 2026 @ 12:42:35.5...	jamil-VirtualBox	PAM: Login sess
Mar 31, 2026 @ 12:42:35.5...	jamil-VirtualBox	PAM: Login sess
Mar 31, 2026 @ 12:42:27.5...	jamil-VirtualBox	PAM: Login sess
Mar 31, 2026 @ 12:42:02.2...	jamil-VirtualBox	PAM: Login sess
Mar 31, 2026 @ 12:40:36.1...	jamil-VirtualBox	PAM: Login sess
Mar 31, 2026 @ 12:40:08.1...	jamil-VirtualBox	PAM: Login sess
Mar 31, 2026 @ 12:40:08.1...	jamil-VirtualBox	PAM: Login sess
Mar 31, 2026 @ 12:40:01.8...	suricata-agent	Apparmor DENIE
Mar 31, 2026 @ 12:39:55.8...	suricata-agent	Apparmor DENIE
Mar 31, 2026 @ 12:39:51.8...	suricata-agent	PAM: Login sess
Mar 31, 2026 @ 12:39:51.8...	suricata-agent	PAM: Login sess
Mar 31, 2026 @ 12:39:51.8...	suricata-agent	Successful sudo

Document Details

Table JSON

```
{
  "_index": "wazuh-alerts-4.x-2026.03.31",
  "_id": "XDVPQ50B18fYcXKA09rc",
  "_score": null,
  "_source": {
    "predecoder": {
      "hostname": "jamil-VirtualBox",
      "program_name": "sudo",
      "timestamp": "Mar 31 08:42:33"
    },
    "input": {
      "type": "log"
    },
    "agent": {
      "name": "jamil-VirtualBox",
      "id": "000"
    },
    "manager": {
      "name": "jamil-VirtualBox"
    },
    "data": {
      "srcuser": "jamil",
      "dstuser": "root",
      "tty": "pts/2",
      "pwd": "/home/jamil/Desktop",
      "command": "/usr/bin/tee -a /var/log/suricata/eve.json"
    },
    "rule": {
      "mail": false,
      "level": 3,
      "pci_dss": [
        "10.2.5",
        "10.2.2"
      ]
    }
  }
}
```